BX-16: RV32I with a 16-bit Datapath and the SVM-16 Profile of RVV

Tönpa Namdak
B-MSX / tronMSX Project
`https://github.com/msx-btron/btron`

November 2026

Abstract

BX-16 replaces both the R800 and any 68000-class host with one RISC-V hart. The architectural ISA is RV32I (XLEN = 32). The ALU, the local data bus, and the vector element width are 16 bits. The vector unit is SVM-16, a profile of RVV 1.0: 32 vector registers, standard OP-V encodings, `vsetvl`, ELEN = 16, VLEN = 128, legal EEW of 8 and 16 only. Widening multiplies produce 32-bit results in a register group. Geometry operations that used to be single VEX opcodes (`VMAT4VEC`, `VCROSS`) are optional `Xsvm` custom instructions, not part of the profile. Z80, R800, and 68000 binaries are not executed. MSX and Amiga-class peripherals may still be attached to the bus. The private R1600 encoding and the 16-register VEX file described in earlier drafts of this note are withdrawn; they are not RVV.

Supersession. Sections below that still name the R800, the R1600, or VEX record the previous proposal. Where they conflict with this abstract, the abstract wins. In particular there is no binary-compatible Z80 mode and no second proprietary core.

Contents

1 Introduction **2**

2 Background and Design Constraints **2**
2.1 The MSX turboR and the R800 . . . 2
2.2 Related Work . . . 3
2.3 Target Workload . . . 3

3 The R1600 Core **3**
3.1 Register File . . . 3
3.2 SIMD Width and Number Format . . . 3
3.3 Instruction Encoding . . . 4

4 VEX — Authoritative Definition for the BX Series **4**
4.1 Architectural State . . . 4
4.2 Selected Operations (BX-16 encoding class) . . . 4
4.3 Source Compatibility with Later RVV Subsets . . . 5

5 System Organisation **5**
5.1 BX-8 3D: R800 + R1600 Coprocessor . . . 5
5.2 BX-16: Dual R1600 . . . 5
5.3 Hardware Block Diagrams (Conceptual) . . . 6

6 Programming Model and Software Compatibility **6**

7 Implementation Context **6**

8 Expected Characteristics and Trade-offs **7**

9 Conclusion **7**

1 Introduction

The MSX turboR, introduced in 1990–1991, remains one of the most elegant examples of evolutionary computer architecture. Its R800 processor retained full Z80 binary compatibility and the classic MSX slot/bus model while delivering a substantial internal 16-bit datapath, improved instruction timing, and modest multiply support. This combination allowed a large existing software base to continue running while new software could exploit the additional performance.

More than three decades later, the same evolutionary pressure exists. Contemporary interest in software-rendered 3D (tinyGL and related fixed-function pipelines), retro-computing research, and FPGA re-implementations raises a natural question: can the turboR lineage be extended further in a way that remains recognisably MSX, preserves cartridge and bus compatibility, and yet provides enough arithmetic and memory bandwidth for interactive low-polygon 3D?

This paper answers that question with two concrete proposals:

- **BX-8 3D** — a conservative, minimal-change extension of the existing turboR. The R800 remains the main CPU; a new 16-bit core (R1600) is added as a tightly-coupled vector/3D coprocessor.
- **BX-16** — a more ambitious dual-processor system in which two R1600 cores operate side-by-side: one as general-purpose CPU, the other as dedicated vector/GPU unit.

Both designs share the same R1600 microarchitecture and the same VEX vector instruction set. The difference lies only in system organisation and the degree of departure from the classic 8-bit external bus.

The work is carried out in the context of the open-source B-MSX / tronMSX project, which already contains a cycle-accurate verifying emulator and an FPGA implementation of a turboR-class machine (BX-8). The proposals are therefore not purely theoretical; they are intended to be implemented by extending existing, publicly available hardware and software artefacts.

VEX, as defined here, is the first concrete realisation of the scalable vector architecture that later appears (under the RVV / Zve* umbrella) on BX-32 and BX-64. The length-agnostic programming style and the element-type hierarchy are intentionally identical, so that the same source kernels compile and run across the whole family.

2 Background and Design Constraints

2.1 The MSX turboR and the R800

The R800 is a microcoded, pipelined processor that executes the Z80 instruction set with significantly improved cycle counts and adds a small number of new instructions (notably `MULUB` and `MULUW`). Internally it already possesses a 16-bit ALU. Externally it presents an 8-bit data bus and the classic MSX slot protocol, guaranteeing that every existing cartridge and peripheral continues to function.

Any credible successor must therefore satisfy three non-negotiable constraints:

1. Binary compatibility with Z80 and R800 code in a well-defined compatibility mode.
2. Electrical and protocol compatibility with the MSX slot bus (at least as an option).
3. A clear, documentable migration path for both assembly-language and compiled software.

2.2 Related Work

Kazuhiko Nishi and collaborators have publicly discussed R900 / R1000 / R1800-class designs that continue the R800 lineage with higher clocks, cache, MMU and floating-point support. Those efforts are valuable. The present proposal differs in emphasis: it prioritises a clean 16-bit programming model and an explicit vector unit sized for 3D transform and lighting workloads, while retaining the possibility of a very high degree of hardware compatibility with existing MSX systems.

2.3 Target Workload

The primary non-legacy workload considered is software 3D rendering of the complexity that tinyGL (and similar fixed-function OpenGL 1.x subsets) can express: low-to-mid polygon counts, simple lighting, perspective-correct or affine texturing, and a software Z-buffer. The architecture must make the transform-and-lighting stage and the inner rasterisation loops substantially faster than a pure R800, without requiring a full modern GPU.

3 The R1600 Core

At the heart of both proposals lies a single new processor core, designated R1600. It is a true 16-bit (with 32-bit scalar registers) orthogonal RISC-like design that can operate in two roles:

- as a general-purpose CPU, or
- as a vector/3D coprocessor.

3.1 Register File

Resource	Width	Notes
Scalar registers R0–R15	32-bit	General purpose; low 16 bits usable for pure 16-bit code
Vector registers VR0–VR15	128-bit	4 × 32-bit lanes (primary) or 8 × 16-bit
SP, PC	32-bit	
SR	32-bit	Flags, interrupt mask, mode bits, VU status

The 16 vector registers of 128 bits each provide 256 bytes of vector state—sufficient to hold a 4×4 matrix (four registers) plus several vertices and temporaries.

3.2 SIMD Width and Number Format

The primary SIMD configuration is 4-wide. Each lane is 32 bits and is interpreted by default as Q16.16 fixed-point (16 integer bits, 16 fractional bits). This format offers a good balance of range and precision for 3D geometry while remaining simple to implement in FPGA logic. Supporting views (8×16-bit, pure integer, later optional FP32) are defined but not required for the initial implementation.

Multiplication follows the rule

$$\text{result}_{\text{Q16.16}} = (a_{\text{Q16.16}} \times b_{\text{Q16.16}}) \gg 16$$

with a 64-bit intermediate product to preserve accuracy. A vector accumulator (VACC) is provided for long multiply-accumulate chains.

3.3 Instruction Encoding

Instructions are primarily 16-bit, with optional 16-bit extension words for immediates and displacements. The major opcode field (bits 15–12) partitions the encoding space:

Bits 15–12	Group	Purpose
0000	System/Control	NOP, HALT, RESET, interrupt control
0001	Scalar ALU (reg-reg)	ADD, SUB, AND, OR, XOR, ...
0010	Scalar Immediate	Arithmetic/logic with immediate
0011	Shift/Rotate	ASL, ASR, LSR, ROL, ROR, ...
0100	Multiply/Divide	MUL, MULU, MULS, DIVU, DIVS
0101	Compare/Test	CMP, TST, CMN
0110	Load/Store (scalar)	Multiple addressing modes
0111	Branch/Jump/Call	Conditional branches, JSR, RTS
1100	VEX Arithmetic	Vector add/sub/mul/madd/dot/cross ...
1101	VEX Memory & Permute	Vector load/store, broadcast, swizzle
1110	VEX Control	VWAIT, VSTATUS, VMAT4VEC, kick
1111	Escape/Extended	Future expansion

A complete bit-field definition for every group is maintained in the project documentation; the most frequently used vector operations appear below.

4 VEX — Authoritative Definition for the BX Series

VEX is the vector extension of the BX family. On BX-16 it is realised natively on the R1600; on BX-32 and BX-64 it is realised by mapping onto the corresponding RVV / Zve* subset while preserving the same programming model.

4.1 Architectural State

- 16 vector registers VR0–VR15, each of width VLEN (on BX-16, VLEN = 128 bits).
- A configuration register pair that is isomorphic to RVV's `vtype` and `vl`:
  - SEW (selected element width) $\in \{8, 16\}$ on BX-16; later $\{8, 16, 32, 64\}$.
  - VL (vector length in elements) set by a `vsetvl`-style instruction.
- A vector accumulator (VACC) for high-precision multiply-accumulate chains.

4.2 Selected Operations (BX-16 encoding class)

Mnemonic	Encoding class	Operation
VADD VRd,VRs,VRt	1100	4-wide addition
VSUB VRd,VRs,VRt	1100	4-wide subtraction
VMUL VRd,VRs,VRt	1100	4-wide Q16.16 multiply
VMADD VRd,VRs,VRt	1100	multiply-accumulate
VDOT Rd,VRs,VRt	1100	scalar dot product
VCROSS VRd,VRs,VRt	1100	3-element cross product
VLD VRd,(Rb)	1101	128-bit vector load
VST VRs,(Rb)	1101	128-bit vector store
VBROADCAST VRd,Rs	1101	broadcast scalar to all lanes
VMAT4VEC VRd,VRs	1110	matrix (VR0–VR3) $\times$ vector
VWAIT	1110	wait until vector unit idle

The special instruction `VMAT4VEC` implements a complete 4×4 matrix-by-vector multiply using the matrix already resident in VR0–VR3, dramatically reducing instruction count for the most common 3D transform.

4.3 Source Compatibility with Later RVV Subsets

- Length-agnostic loops written against VL and SEW compile unchanged when the host becomes RV32 or RV64.
- The domain-specific accelerators (VMAT4VEC, VCROSS, ...) are either retained as custom opcodes in the RVV custom-0/1 space or expanded by the compiler into short standard RVV sequences.
- Element-type hierarchy, saturating arithmetic and widening multiplies are identical across the family.

This is the precise mechanism that allows the BX series to claim a single scalable vector programming model from 16-bit to 64-bit.

5 System Organisation

5.1 BX-8 3D: R800 + R1600 Coprocessor

In the BX-8 3D configuration the existing R800 remains the main CPU and continues to own the classic MSX slot bus. An R1600 core is attached as a tightly-coupled coprocessor. Communication occurs through:

- a shared memory window (typically 4–16 KB, dual-ported or DMA-accessible),
- a command FIFO,
- status and doorbell registers.

The R800 issues work by writing parameters and a command packet into the shared region and then writing a kick register. It may poll for completion or receive an interrupt. All existing MSX software runs unchanged; new software detects the presence of the vector unit and offloads transform, lighting and batch vertex work.

This organisation satisfies the original BX-8 3D design goal: maximal compatibility with minimal changes to the surrounding system logic.

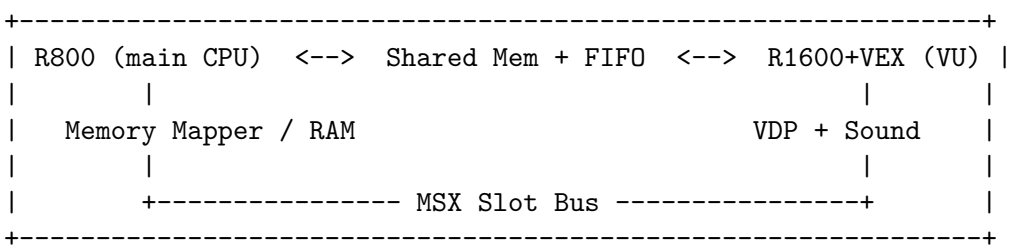
5.2 BX-16: Dual R1600

In the full BX-16 system two identical R1600 cores are instantiated. One runs general-purpose code (operating system, game logic, classic MSX compatibility layer). The second is dedicated to vector and simple raster workloads. A high-bandwidth 16-bit local interconnect, shared memory, and the same command-FIFO protocol used in BX-8 3D allow the two cores to cooperate efficiently.

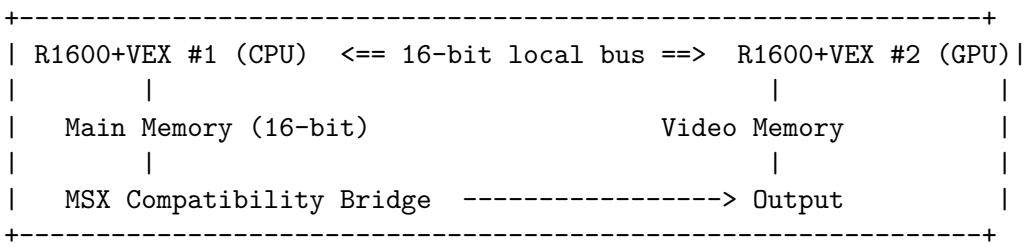
A compatibility bridge presents a classic 8-bit MSX slot interface to the outside world, so that existing cartridges can still be used. Internally the machine is a clean 16-bit dual-processor system.

5.3 Hardware Block Diagrams (Conceptual)

BX-8 3D



BX-16



6 Programming Model and Software Compatibility

A single high-level 3D library (or a tuned tinyGL back-end) can target both machines. The portable layer performs matrix construction, scene management and high-level rendering calls. Only the lowest-level kernels—vertex transform, matrix–vector products, and block fills—are written against the VEX instruction set or the command-FIFO interface.

On BX-8 3D the main thread runs on the R800 (or its 16-bit extended form) and kicks the R1600. On BX-16 the same source can be compiled to native R1600 code for the primary core while the secondary core executes the vector kernels. Detection of the exact configuration is performed at start-up via a status register, allowing a single binary to adapt.

Assembly-language programmers retain two clean models:

- an R800-like additive style for BX-8 3D (new vector operations appear as coprocessor or extended opcodes),
- a pure orthogonal 16-bit style for native BX-16 code.

Both models are fully documented with bit-accurate encodings, assembler syntax and calling conventions.

7 Implementation Context

The proposals are developed inside the B-MSX / tronMSX open-source effort:

- **Verifying emulator** (cycle-accurate reference): https://github.com/msx-btron/btron/tree/main/src/emulators/msx
- **BX-8 FPGA implementation** (Bluespec SystemVerilog): https://github.com/msx-btron/btron/tree/main/src/emulators/bx8/bsv
- **BTRON host environment** — a lightweight host layer used for development, testing and future higher-level services.

The existing BX-8 BSV sources already contain a clean modular structure (CPU, memory, VDP, audio, bus). Extending them with an R1600 core and the shared-memory/FIFO interface is a well-scoped engineering task. The verifying emulator provides a reference against which both functional and timing correctness can be checked.

8 Expected Characteristics and Trade-offs

Property	BX-8 3D	BX-16
MSX cartridge compatibility	Native, excellent	High (via bridge)
Binary compatibility (Z80/R800)	Full	Full (compatibility mode)
3D transform performance	Good	High
Implementation complexity	Moderate	Higher
Silicon / FPGA resource use	R800 + one R1600	Two R1600
Programming model purity	Hybrid	Clean 16-bit

BX-8 3D maximises compatibility and minimises risk; it is the natural next step for an existing turboR-class FPGA design. BX-16 trades some purity of the external bus for a cleaner internal architecture and substantially higher sustained 3D throughput. Both remain within the philosophical bounds of an MSX evolutionary path.

9 Conclusion

The MSX architecture has always favoured careful, compatibility-preserving evolution over radical breaks. The R800 itself was an embodiment of that principle. The BX-8 3D and BX-16 proposals continue the same tradition into the era of software 3D.

By defining a single reusable 16-bit core with a practical vector extension (VEX), and by offering two system organisations that differ only in the degree of coupling and bus width, we obtain a scalable family: a highly compatible turboR successor on one hand, and a more powerful dual-processor hybrid on the other. Both can share algorithms, libraries and even substantial amounts of implementation IP.

VEX, as specified in this note, is the authoritative foundation of the vector architecture used throughout the BTRON BX series. The same programming model scales cleanly to the Zve* and full RVV implementations of BX-32 and BX-64.

The designs are offered for discussion, refinement and, we hope, eventual realisation within the open-source MSX research community and beyond.

Acknowledgements

This work builds directly on the MSX architectural legacy created by ASCII Corporation and the many engineers who contributed to the turboR. The open-source BX-8 and tronMSX code bases provide the concrete foundation that makes a proposal of this concreteness possible.

References

[1] MSX Technical Handbook / MSX Datapack (ASCII / various editions).

[2] ASCII Corporation, R800 documentation and turboR hardware manuals (1990–1991).

[3] TinyGL / related software OpenGL 1.x subset implementations.

[4] B-MSX / tronMSX project repositories:
https://github.com/msx-btron/btron